
PRIMER · V1

The GitHub You Actually Need

Not a general tutorial. Just the concepts the App Factory runs on, why each one is there, and what the confusing error messages are really telling you.

For Keshav · Written 10 August 2026

Every example here is from your own repository. Every error is one you've already hit.

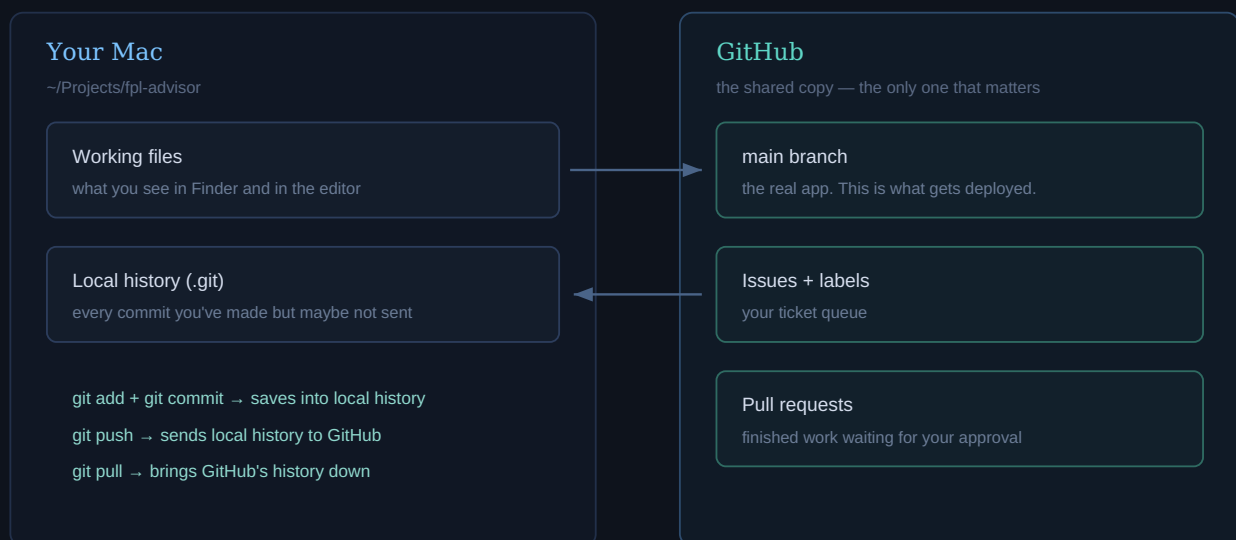
Why you need any of this

GitHub is doing three separate jobs in your system, and they're easy to confuse because they live behind the same website.

Storage	It holds the code, and every version of it that ever existed.
The queue	Issues and their labels are how you tell the overnight agents what to build.
The gate	Pull requests are where work waits for you, and merging is the one action no agent can take.

You don't need to be good at git. You need to understand about eight ideas well enough that nothing surprises you at 8am.

The mental model



Almost every confusing git error comes from the gap between these two boxes.

The single most useful idea: your Mac and GitHub are two separate copies. Nothing moves between them unless you say so. When something behaves strangely, ask "which copy am I looking at?" and the answer usually falls out.

The eight ideas

1 · Repository

A project folder that git is tracking. Yours is `KeshavPeri/fpl-advisor`. It exists twice: on your Mac at `~/Projects/fpl-advisor`, and on GitHub. One repo per app, deliberately — each has its own hosting and its own release rhythm.

2 · Commit

A saved snapshot with a message explaining why. Not a save button — a save button with a note attached, and the note is the point. Six months on, `git log` is a written record of what happened and why.

A commit lives only on your Mac until you push it. This catches people constantly.

3 · Branch

A parallel line of work. `main` is the real app — what Vercel deploys to production. Everything else is temporary.

Your agents work on branches named `claude/ticket-3-home-screen-tagline` and so on. That prefix isn't decorative: the platform only reliably accepts pushes to `claude/`-prefixed branches, so the convention matches the restriction rather than fighting it.

Branches are cheap and disposable. Delete them after merging. A leftover `claude/ticket-*` branch with commits on it will make the next run think a previous run died.

4 · Push and pull

`git push` sends your local commits to GitHub. `git pull` brings GitHub's down. **The agents only ever see what's on GitHub** — a cloud run clones the repo fresh from the default branch every time.

This has already bitten you once

You committed rewritten agent definitions and the routine kept using the old ones — because the commit had landed on a leftover feature branch, not on `main`, and hadn't been pushed. After changing anything the agents rely on, verify with `git log origin/main --oneline -3`. If your change isn't in that list, the agents cannot see it.

5 · Issue

A ticket. In normal projects it's a to-do note; here it's a work order that an agent reads and executes. Which is why the template has a mandatory definition-of-done section — an agent can't infer what "finished" means.

6 · Label

A coloured tag on an issue. In your system labels aren't decoration — they *are* the state machine. Exactly one `status:` label per open issue, and that label determines what happens tonight.

The dangerous property of labels

Applying a label that doesn't exist **creates it silently**. There's no error and no warning. Type `status:redy` and GitHub cheerfully makes a new label, your ticket carries a state nothing queries, and it vanishes from the pipeline. Always pick from the dropdown; never type a label name.

7 · Pull request

The name is unhelpful. A pull request is **a proposal to merge one branch into another**, with a discussion attached. Your agents open one per finished ticket, from their `claude/ticket-*` branch into `main`.

A **draft** PR is one explicitly marked not-ready-to-merge. Agents always open drafts. Converting it with **Ready for review** is a small deliberate act by you — which is exactly the point.

Tabs worth knowing:

- **Conversation** — the description (your review packet) and comments. This is where you live.
- **Files changed** — the combined difference between the branch and `main`. Green added, red removed.
- **Commits** — the individual steps. Rarely useful.

Files changed vs. Commits — a real distinction

During the smoke test, one commit accidentally deleted your entire decisions log and a later commit restored it. Looking at that *commit* showed a terrifying wall of red. Looking at **Files changed** — the net effect — showed six harmless added lines. When you want to know "what will actually change if I merge this," it's always Files changed.

8 · Merge

Taking the branch's changes into `main`. Vercel notices and deploys to production within a minute or two. This is the only step no agent can perform.

GitHub offers three ways. You only need two:

Create a merge commit	Keeps every individual commit from the branch. Fine as a default.
Squash and merge	Collapses the whole branch into one commit on <code>main</code> . Use this when the branch has messy intermediate history — like a mistake and its correction.
Rebase and merge	Ignore it.

Bonus · "Closes #6"

When a PR description contains `Closes #6`, merging it automatically closes issue 6. That's the mechanism that moves your ticket to Done without anyone touching a label. If an issue doesn't close itself on merge, the agent left that line out — which is a bug in the prompt, not something to fix by hand.

Errors you've already seen

"Unable to create '.git/index.lock': File exists"

Git locks the repo while it works. If a process dies mid-operation, the lock file is left behind and every later command refuses to run. Nothing is broken and nothing is lost.

```
rm ~/Projects/fpl-advisor/.git/index.lock
```

Only do this when you're sure no git command is genuinely running.

"Updates were rejected because the remote contains work that you do not have locally"

GitHub has commits you don't. Usually because you merged a PR in the browser. Git won't overwrite them, which is correct behaviour.

```
git pull --rebase origin main
git push origin main
```

`--rebase` replays your commits on top of what GitHub has, instead of creating a merge commit for what's usually a one-file change.

"error: unable to delete 'X': remote ref does not exist"

You tried to delete a branch on GitHub that was already deleted. Your Mac was holding a stale pointer to it. Harmless.

```
git remote prune origin
```

"error: branch 'claude/ticket-5-...' not found"

You tried to delete a branch locally that only ever existed on GitHub — because a cloud run created it, not your Mac. Nothing to do.

"nothing added to commit but untracked files present"

There's a new file git isn't tracking yet. `git add .` won't include it if it's ignored. This is how `.claude/settings.json` sat uncommitted for weeks while everyone assumed it was in the repo.

Commands worth knowing

Roughly ten will cover everything you do. The first four are read-only — they change nothing, so run them freely whenever you're unsure.

```
# Read-only. Safe. Run these when confused.
git status                # what's changed, which branch
git branch                # local branches, * marks current
git log --oneline -5      # last 5 commits here
git log origin/main --oneline -5 # last 5 commits on GitHub

# Everyday
git add <file>            # stage a specific file
git commit -m "why this change" # save with a message
git push origin main      # send to GitHub
git pull                  # bring GitHub's changes down

# Occasional
git checkout main         # switch to main
git remote prune origin   # clear stale branch pointers
git branch -D <branch>    # delete a local branch
```

A habit worth forming

Run `git status` before and after anything that changes state. It tells you the branch you're on, what's modified, and whether you're ahead of GitHub. Most git confusion is really "I didn't know which branch I was on."

The parts specific to your system

Why the agents can't touch a project board

GitHub Projects — the kanban-board feature — is reachable only through an API called GraphQL. Cloud runs sit behind a security proxy that blocks GraphQL and permits only repository-scoped requests. Both routes are closed, which was confirmed by running two diagnostic probes rather than by reading documentation.

Labels are ordinary repository data, so they work. They also do something a board never could: a blocked ticket gets a comment, and a comment sends you a phone notification.

Why branches are named `claude/...`

Cloud runs push reliably only to branches with that prefix. The convention matches the platform's restriction rather than fighting it.

Why the agents can't push to `main`

By design, and it's load-bearing. It means no agent can change production without passing through a pull request and your merge. It also has a side effect worth understanding: when the orchestrator writes to the decision log, that write rides the ticket's branch and only reaches `main` when you merge. For a blocked ticket — which never merges — decisions are recorded in the issue comment instead.

What the Claude GitHub App is for

An installed integration granting the cloud runs access to your repository's issues, pull requests and deployment status. Without it, runs can clone the code but can't read PR data — so every review packet comes back saying "Preview: pending".

Glossary

branch	A parallel line of work. <code>main</code> is the real one.
clone	Downloading a full copy of a repository.
commit	A saved snapshot with a message.
default branch	The branch GitHub shows first, and the one cloud runs clone. Yours is <code>main</code> .
diff	The difference between two versions. Green added, red removed.
draft PR	A pull request explicitly marked not ready to merge.
fast-forward	A merge with nothing to reconcile — git just moves the pointer.
fetch / pull	Fetch downloads GitHub's changes. Pull downloads and applies them.
issue	A ticket. Here, a work order for an agent.
label	A tag on an issue. Here, the pipeline's state machine.
merge	Bringing a branch's changes into another branch.
origin	Git's nickname for "the copy on GitHub".
PR	Pull request. A proposal to merge, with discussion attached.
push	Sending local commits to GitHub.
rebase	Replaying your commits on top of someone else's.
remote	A copy of the repo somewhere else. Yours is GitHub.
squash	Collapsing several commits into one.
staging	The waiting area between editing and committing. What <code>git add</code> fills.
untracked	A file git has never been told to watch.

Questions

Can I break something permanently?

Almost never. Git keeps everything committed, and anything pushed to GitHub is recoverable. The two genuinely destructive things are deleting the repository itself, and force-pushing over history — you'll never need either. Uncommitted work is the one real vulnerability, which is why agents are told to commit early and often.

Do I need the command line at all?

Barely. Reviewing, merging, issues, labels and branch deletion are all in the browser and on your phone. Terminal is only needed for linting tickets and for occasionally pushing a change to an agent definition.

What if I edit a file on my Mac and the agent edits the same file overnight?

You'll get a merge conflict when you pull — git asking you to choose. Easily avoided: don't hand-edit files inside an app repo while a ticket for them is queued. Edit the agent definitions and docs, and let the pipeline own the source code.

Why is the decision log on a branch instead of `main`?

Because agents can't push to `main`. Log entries ride the ticket's branch and land when you merge. It's a consequence of the safety property, not an oversight.

Should I keep the GitHub mobile app?

Yes. Your whole morning review is designed around it — notification, tap the preview, read the packet, merge. It's the closest thing this system has to a user interface.

What's the difference between closing an issue and deleting it?

Closing is normal and reversible; the issue stays searchable, which matters because useful reasoning often lives in the comments. Deleting is permanent. Always close.

You don't need to master git. You need to know that your Mac and GitHub are separate copies, that `main` is the real app, that labels drive the queue, and that merging is yours alone. Everything else you can look up.